

Patrones

1_ Utilice el patrón “Adapter” para convertir la interfaz de una clase externa en una interfaz que el cliente espera, haciendo que la interfaz actúe como traductor.

2_ Utilice “Template Method” ya que los empleados tenían comportamientos en común, con este patrón maximizo la reutilización de código y evito código repetido.

3_ Use el patrón “Adapter” para integrar la clase “VideoStream”, creando la clase “VideoStreamAdapter” que tiene una variable de “VideoStream”, es una extensión de la clase “Media” y sabe responder al mensaje “Play()”

4_ Use el patrón “Composite” para tratar de la misma forma a las topografías simples (agua, tierra, pantano) y a las composiciones de objetos (mixta). También apliqué Factory Method para simplificar la creación de las clases y solo tener 2 tipos de topografías.

5_ Use Factory Method para la creación de las sustancias químicas, porque siempre utilizan la misma fórmula para su creación.

6_ Use el patrón Builder para la creación de sándwich, porque todos siguen la misma serie de pasos para su creación.

7_ Use Factory Method para desacoplar la creación de la funcionalidad de los productos, en este caso el cliente solo pide el producto que necesita, sin importar cómo se construye. La única desventaja de este patrón es la complejidad estructural que presenta.

8_ Use el patrón state para representar los cambios de estado y evitar condicionales.

9_ Use el patrón strategy porque te da la posibilidad de cambiar los algoritmos en tiempo de ejecución.

10_ Use el patrón state para representar los estados de la calculadora y que estos sepan responder a los mensajes correspondientes. También uso Template Method para organizar los estados de cálculo y evitar código repetido.

11_ Use el patrón composite para tratar de la misma forma a los archivos y directorios.

12_ Use el patrón strategy para representar las distintas políticas de cancelación. Ventajas de diseño:

- **Bajo acoplamiento:** El auto no sabe cómo se calcula el reembolso, solo sabe que alguien lo hace por él.
- **OCP(open/close principle):** El diseño es abierto para extensión, se pueden agregar nuevas políticas de cancelación, sin necesidad de modificar el funcionamiento del programa.

14_ Uso patrón builder para el armado de pc, porque cada pc cuenta con una serie de pasos para su construcción.

15_ Uso patrones Strtegy y adapter. Strategy para darle la posibilidad al mensajero de cambiar el algoritmo de cifrado, y el adapter para que las clases de cifrado sepan responder a los mensajes de la interfaz de cifrado.

16_ Uso el patrón state porque el comportamiento de la excursión varía según su estado interno. Sin este patrón la excursión tendría muchos if/else para verificar el estado actual, lo que dificulta la extensión y el mantenimiento.

17_ Utilizo un strategy para darle la posibilidad al usuario de poder intercambiar crc calculator y el patrón adapter para que la clase "4gConnection" sepa responder a los mensajes de la interfaz "Connection".

18_ Utilizo el patrón Factory para la creación de personajes, porque cada personaje siempre se crea con una armadura, arma, habilidades y 100 puntos de vida. Para representar las armas y armaduras utilice doble dispatch.

19_ Utilizo el patrón decorator para armar los mensajes de manera personalizada y poder modificarlos en tiempo de ejecución. Uso Template Method para evitar código repetido.

20_ Use el patrón proxy para generar un intermediario entre la base de datos y el usuario, el intermediario controla que sólo los usuarios autenticados puedan interactuar con la base de datos.

21_ Uso el patrón decorator para agregar nuevas funcionalidades y el patrón adapter para poder cambiar a grados celsius sin modificar la clase "HomeWeatherStation".

23_ Uso el patrón Null Object para reemplazar referencias nulas por un objeto que implementa la interfaz y su comportamiento es no hacer nada, pero evita preguntas explícitas por null.

24_ Uso el patrón Proxy para trabajar que el usuario no interactue directamente con los repositorios y para almacenar los post en la clase proxy, de esta forma solo busco los post cuando es realmente necesario, ya que la consulta en la base de datos es costosa.

25_ Uso el patrón Strategy para darle la posibilidad al afiliado de cambiar de plan médico y Template Method para no repetir código.

26_ Uso el patrón Template Method para calcular el costo de las prendas y Composite para dar la posibilidad al banco de combinar prendas.

27_ Opción A

- a. ¿La variable estado, implementa correctamente el patrón State? Responda si o no.

Respuesta: No. Un string no debe representar un estado, debe ser reemplazo por una clase abstracta llamada “estado” y tener una clase concreta por cada estado posible que extienda de la clase “estado”, de esta forma cada estado sabe responder a los mensajes solicitados adecuadamente.

- b. ¿Respecto del patrón State, es correcto que la misma clase PaqueteViaje implemente el comportamiento a realizar según el estado? Responda si o no.

Respuesta: No, debe ser responsabilidad del estado saber responder a los mensajes según su estado actual.

- d. ¿La variable estrategia, implementa correctamente el patrón Strategy? Responda si o no.

Respuesta: No, se llama “estrategia” pero funciona como un patrón state. En el patrón strategy la estrategia no debe cambiar según el estado actual, debe cambiar solo cuando el cliente lo decide

Opción B

- a. ¿La variable estado, implementa correctamente el patrón State?. Responda si o no.

Respuesta: Si.

- b. ¿Respecto del patrón State, es correcto crear diferentes clases para implementar el comportamiento a realizar según el estado?. Responda si o no.

Respuesta: Si, es el propósito del state.

- d. ¿Existe en el código alguna implementación del patrón Strategy? Responda si o no.

Respuesta: Si, está presente en la plataforma de pago.

- e. Si respondió Sí indique donde está implementado el patrón. Indique los roles de cada clase.

Respuesta: El patrón Strategy está implementado para resolver el cobro del paquete, separando el algoritmo de pago de la clase principal. Los roles son los siguientes:

- **Context (Contexto):** La clase “PaqueteViaje”. Mantiene una referencia a la estrategia y le delega la responsabilidad de cobrar llamando al método correspondiente.
- **Strategy (Interfaz Estrategia):** La interfaz “PlataformaPago”. Define el método “pagar()” que todas las estrategias de pago deben respetar de forma polimórfica.